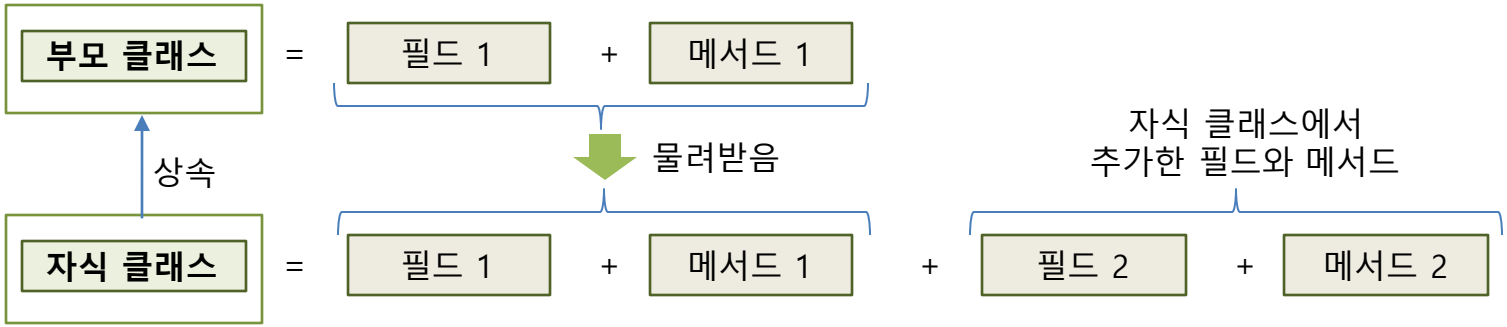
12강. 상속 - I

- 상속 개념
- 상속 문법의 이해

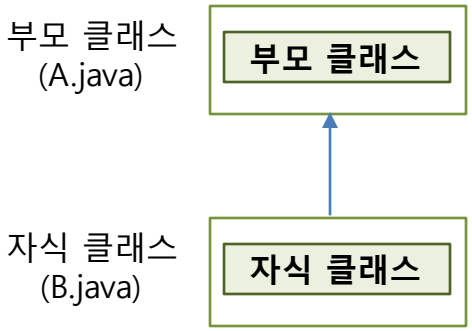
12-1. 상속 개념

- 상속(Inheritance): 부모가 자식에게 물려주는 행위로 객체 지향 프로그램에서도 부모 클래스의 필드와 메서드를 자식 클래스에 물려줄 수 있다.



- 상속은 이미 잘 개발된 클래스를 재사용하여 새로운 클래스를 만들기 때문에 중복되는 코드를 줄여 개발 시간을 단축시키고, 프로그램의 완성도를 높인다.

- 예) B클래스는 A가 가진 필드와 메서드를 전부 작성하는 것보다 A를 상속 받고 필요한 필드와 메서드를 추가 작성하는 것이 효율적이다.




```
public class A {
  int field1;
  void method1() { ... }
}
```

A를 상속

```
public class B extends A {
  String field2;
  void method2() { ... }
}
```

12-1. 상속 개념

```
B b = new B();
b.field1 = 10;
b.method1();

b.field2 = "홍길동";
b.method2();
```

A로부터 물려 받은 필드와 메서드

B가 추가한 필드와 메서드

- 상속의 또 다른 이점
  - 클래스의 수정을 최소화 할 수 있다.(부모 클래스를 수정하면 상속 받은 모든 자식 클래스가 수정되는 효과)

12-2. 상속 문법의 이해

- **클래스 상속(java_12_1_ex1_inheritance)**

➢ 현실 세계에서 상속: 부모가 자식을 선택해서 물려준다.

➢ 자바 상속: 자식이 상속 받을 부모를 선택한다. 즉, 자식 클래스를 선언할 때 어떤 부모로 상속 받을 것인지를 결정한다.

➢ 상속 선언 방법: 자식 클래스 명 뒤에 extends를 붙이고, 그 뒤에 상속 받을 부모 클래스를 기술한다.

```
public class 자식 클래스 extends 부모 클래스 {

}
```

➢ 자바 클래스 상속 특징: 다중 상속 불가(단일 상속만 가능하므로 extends 뒤에서 단 하나의 부모 클래스만 올 수 있다.)

```
public class 자식 클래스 extends 부모 클래스1, 부모 클래스2 {

}
```

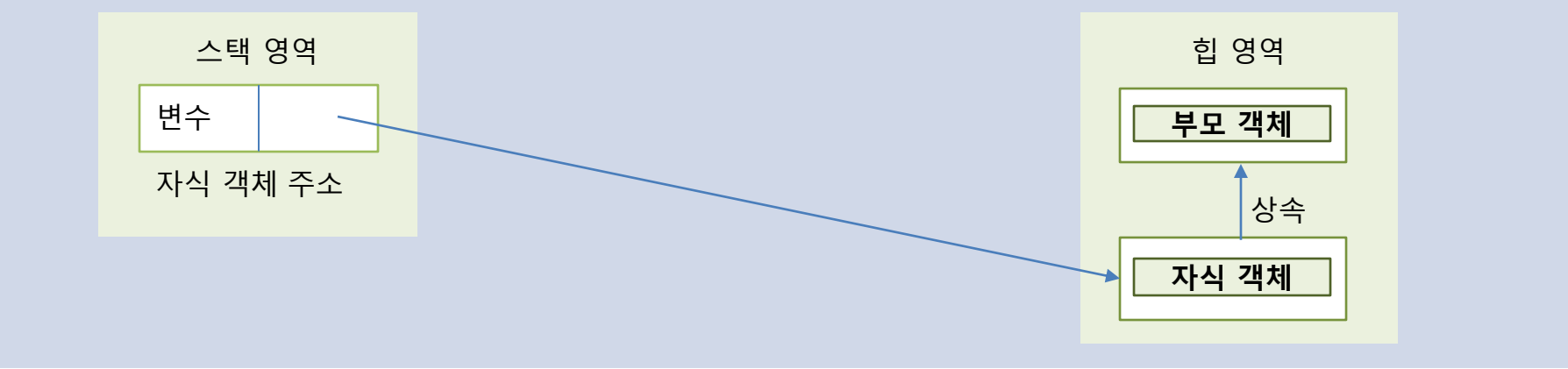
12-2. 상속 문법의 이해

● 부모 생성자 호출

➢ 현실에서 부모 없는 자식이 없듯이 자바에서도 자식 객체를 생성하면 부모 객체가 먼저 생성된 다음에 자식 객체가 생성이 된다.

```
자식 클래스 변수 = new 자식 클래스( );
```

(메모리 표현)



➢ 모든 객체는 생성자를 호출해야만 생성된다. 부모 객체도 예외가 아니며, 부모 생성자는 자식 생성자의 맨 첫 줄에 숨겨져 있는 super()에 의해 호출된다.

```
// 자식 생성자 선언
public class 자식 클래스 {
  super();
  …
}
```

12-2. 상속 문법의 이해

● 부모 생성자 호출

- super()는 컴파일 과정에서 자동 추가되며, 부모의 기본 생성자를 호출한다. 만약 기본 생성자가 없다면 에러가 발생한다.
- 부모 클래스에 기본 생성자가 없고 매개변수를 갖는 생성자만 있다면 super(매개값,...) 코드를 직접 넣어 일치하는 부모 생성자를 호출한다.

```
// 자식 생성자 선언
public class 자식 클래스 {
  super( 매개값, ... );
  ...
}
```

12-2. 상속 문법의 이해

- **부모 생성자 호출 – 부모 클래스가 기본 생성자를 가진 경우**

```
public class Phone {
    // 필드 선언
    public String model;
    public String color;

    // 기본 생성자 선언
    public Phone() {
        System.out.println("Phone() 생성자 실행");
    }
}
public class SmartPhone extends Phone {
    // 자식 생성자 선언
    public SmartPhone(String model, String color) {
        super();                                    // 생략 가능함(컴파일 시 자동 추가)
        this.model = model;
        this.color = color;
        System.out.println("SmartPhone(String model, String color) 생성자 실행");
    }
}
public class SmartPhoneExam {
    public static void main(String[] args) {
        // SmartPhone 객체 생성
        SmartPhone myPhone = new SmartPhone("갤럭시 S23", "검정색");

        System.out.println("모델: " + myPhone.model);
        System.out.println("색상: " + myPhone.color);
    }
}
```

12-2. 상속 문법의 이해

- **부모 생성자 호출 – 부모 클래스가 매개 변수가 있는 생성자를 가진 경우**

```
public class Phone {
    // 필드 선언
    public String model;
    public String color;

    // 매개변수를 가지고 있는 생성자 선언
    public Phone(String model, String color) {
        this.model = model;
        this.color = color;

        System.out.println("Phone(String model, String color) 생성자 실행");
    }
}
public class SmartPhone extends Phone {
    // 자식 생성자 선언
    public SmartPhone(String model, String color) {
        super(model, color);                        // 매개 값을 가진 생성자 직접 호출

        System.out.println("SmartPhone(String model, String color) 생성자 실행");
    }
}
public class SmartPhoneExam {
    public static void main(String[] args) {
        // SmartPhone 객체 생성
        SmartPhone myPhone = new SmartPhone("갤럭시 S23", "검정색");

        System.out.println("모델: " + myPhone.model);
        System.out.println("색상: " + myPhone.color);
    }
}
```

12-2. 상속 문법의 이해

● 메서드 재정의(java_12_2_ex4_inheritance)

- 부모 클래스의 모든 메서드가 자식 클래스에 맞게 설계되어 있다면 가장 이상적인 상속이지만, 어떤 메서드는 자식 클래스에 적합하지 않을 수 있고, 이를 자식 클래스에서 재정의하여 사용하는 데 이를 메서드 오버라이딩(overriding)이라고 한다.
- 메서드 오버라이딩은 상속된 메서드를 자식 클래스에서 재정의하는 것을 말하며, 메서드가 오버라이딩 되었다면 부모 메서드는 숨겨지고 자식 메서드가 우선적으로 사용된다.

```
class Parent {
  void method1() { ... };
  void method2() { ... };
}
```

상속

```
class Child extends Parent {
  void method2() { ... };
  void method3() { ... };
}
```

```
class ChildExam {
  public static void main(String[] args) {
    Child child = new Child();

    child.method1();

    child.method2();

    child.method3();

  }
}
```

X

O

< 메서드 오버라이딩 작성 규칙>

- 부모 메서드 선언부(리턴 타입, 메서드 명, 매개변수)와 동일
- 접근 제한을 더 강하게 오버라이딩 할 수 없다.(public->private 변경 불가)
- 새로운 예외를 throws 할 수 없다.

12-2. 상속 문법의 이해

- **메서드 재정의 – 사용 예**

```
public class CircleAreaParent {
    public double circleArea(double r) {
        System.out.println("CircleAreaParent 객체의 circleArea() 실행");

        return 3.14159 * r * r;
    }
}
```

```
public class CircleAreaChild extends CircleAreaParent{
    @Override                    // 컴파일 시 정확히 오버라이딩 되었는지 체크해 줌(생략 가능)
    public double circleArea(double r) {
        System.out.println("CircleAreaChild 객체의 CircleArea() 실행");
        return Math.PI * r * r;
    }
}
```

```
public class CircleAreaMain {
    public static void main(String[] args) {
        int r = 10;

        CircleAreaParent circleAreaParent = new CircleAreaParent();
        System.out.println("원 면적: " + circleAreaParent.circleArea(r));
        System.out.println();

        CircleAreaChild circleAreaChild = new CircleAreaChild();
        System.out.println("원 면적: " + circleAreaChild.circleArea(r));
    }
}
```

12-2. 상속 문법의 이해

- **부모 메서드 호출(java_12_2_ex5_inheritance)**

➢ 메서드를 재정의(Override)하면 부모 메서드는 숨겨지고 자식 메서드만 사용되기 때문에 부모 메서드의 일부만 변경된다 하더라도 중복된 내용을 자식 메서드도 가지고 있어야 한다. 이 문제는 자식 메서드 내에서 부모 메서드를 호출하여 해결할 수 있는데 이때 사용하는 키워드가 super()이다. super키워드와 도트(.) 연산자를 사용하면 숨겨진 부모 메서드를 호출할 수 있다.

```
class Parent {
  public void method() {
    // 작업처리1
  };
}
```

상속

부모 메서드 호출

```
class Child extends Parent {
  @Override
  public void method () {
    super.method();

    //작업처리2
  };
}
```

➢ super.method() 위치는 작업처리2의 전후에 어디든지 올 수 있다. 이 방법은 부모 메서드를 재사용함으로써 자식 메서드의 중복 작업 내용을 없애는 효과를 가져온다.

12-2. 상속 문법의 이해

- **부모 메서드 호출 – 예제**

1. Airplane 클래스(parent)는 "착륙합니다", "일반 비행합니다", "이륙합니다" 메서드가 있다.
2. SupersonicAirplane 클래스(child)는 flyMode(NORMAL, SUPERSONIC)에 따라 "초음속 비행합니다" 가 메서드 재정의 되어 있다.
3. 위 조건으로 부모 메서드 호출 방식을 사용하여 아래의 실행 결과가 출력되도록 프로그램 예제를 만드세요.

```
착륙합니다.
일반 비행합니다.
초음속 비행합니다.
일반 비행합니다.
착륙합니다.
```

12-2. 상속 문법의 이해

- **final클래스와 final메서드**

➢ 클래스를 선언할 때 final 키워드를 class 앞에 붙이면 최종 클래스이므로 상속할 수 없다. 즉, final클래스는 부모 클래스가 될 수 없어 자식 클래스를 만들 수 없다.

(사용 형식)

```
public final class 클래스 { ... }
```

➢ 대표적인 예가 String클래스이다.

```
// String class 선언
public final class String { ... }

Public class NewString extends String { ... }    // final로 선언되어 있어 자식 클래스를 만들 수 없다.
```

(예시)

```
package javaex.inheritance;

public final class User { ... }
```

```
package javaex.inheritance;

public class AdminUser extends User { ... }
```

12-2. 상속 문법의 이해

● final클래스와 final메서드

➢ 메서드를 선언할 때 final 키워드를 붙이면 최종적인 메서드이므로 오버라이딩 할 수 없는 메서드가 된다. 즉, 부모 클래스를 상속해서 자식 클래스를 선언할 때 부모 클래스에 선언된 final 메서드는 재정의할 수 없다.

(사용 형식)

```
public final 리턴타입 메서드(매개변수, ...) { ... }
```

(예시)

```
package javaex.inheritance;

public class Car {
  public int speed;                    // 필드 선언

  public void speedup() {                    // 메서드 선언
    speed += 1;
  }

  public final void stop() {                 // final 메서드 선언
    System.out.println("차를 멈춤");
    speed = 0;
  }
}
```

12-2. 상속 문법의 이해

- **final클래스와 final메서드**

(예시)

```
package javaex.inheritance;

public class SportsCar extends Car {
  @Override
  public void speedup() {                    // 메서드 선언
    speed += 60;
  }
                                  X
  @Override
  public void stop() {
    System.out.println("스포츠카를  멈춤");
    speed = 0;
  }

}
```

오버라이딩 할 수 없음